Training Deep Neural Networks

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

Emmanuel Djegou, PhD
emmanueldjegou5@gmail.com

Course Outline

Module 6 — Regularization

1. What is regularization?
2. Parameter norm penalties
3. L2 regularization (weight decay)
4. L1 regularization and sparsity
5. Dataset augmentation
6. Early stopping
7. Dropout
8. Ensemble methods and bagging
9. Label smoothing

Module 7 — Optimization

10. Learning vs. pure optimization
11. Optimization challenges
12. SGD algorithm
13. Momentum and Nesterov
14. Parameter initialization
15. AdaGrad
16. RMSProp
17. Adam
18. Choosing an optimizer

What Is Regularization?

Definition

Regularization is any modification to a learning algorithm intended to reduce its **generalization error** but not its training error.

The core trade-off:

- Regularization trades *increased bias* for *reduced variance*
- An effective regularizer reduces variance significantly without overly increasing bias
- **Goal:** move a model from the overfitting regime (high variance) into the regime that matches the true data-generating process

In practice, the best deep learning model is a **large, heavily regularized model** — not a small model.

Forms of regularization:

- **Norm penalties:** add $\Omega(\theta)$ to the objective
- **Hard constraints:** restrict parameters to a feasible set
- **Data augmentation:** create synthetic training examples
- **Early stopping:** halt before overfitting begins
- **Dropout:** randomly remove units during training
- **Ensemble methods:** combine many models
- **Label smoothing:** soften hard training targets

Parameter Norm Penalties: The Framework

Regularized objective

$$\tilde{J}(\theta; X, y) = J(\theta; X, y) + \alpha \Omega(\theta)$$

- $\alpha \in [0, \infty)$: regularization strength hyperparameter
- $\alpha = 0$: no regularization
- Larger α: stronger regularization, smaller weights

Important convention:

We penalize only the **weights** w, not the biases.

- Each weight specifies how two variables interact and requires observing both variables in a variety of conditions
- Each bias controls only a single variable and requires little data to fit
- Regularizing biases can cause underfitting

Minimising $\tilde{J}$ simultaneously:

1. Fits the training data well (reduces J)
2. Keeps the weights small (reduces Ω)

Bayesian interpretation:

Adding $\alpha \Omega(\theta)$ is equivalent to MAP inference with a prior $p(\theta) \propto e^{-\alpha \Omega(\theta)}$.

- $\Omega = \frac{1}{2}\|w\|_2^2 \Leftrightarrow$ Gaussian prior
- $\Omega = \|w\|_1 \Leftrightarrow$ Laplace prior

L2 Regularization (Weight Decay): Mechanism

L2 penalty and regularized objective

$$\Omega(w) = \frac{1}{2}\|w\|_2^2, \qquad \tilde{J}(w) = J(w) + \frac{\alpha}{2} w^\top w$$

Gradient of the regularized objective:

$$\nabla_w \tilde{J} = \alpha w + \nabla_w J$$

One gradient step:

$$w \leftarrow w - \epsilon\big(\alpha w + \nabla_w J\big) = (1 - \epsilon\alpha)\, w - \epsilon\, \nabla_w J$$

The weight vector is **multiplicatively shrunk** by $(1 - \epsilon\alpha)$ before each gradient update. This is "weight decay."

Effect over the full trajectory:

Using the Hessian eigendecomposition $H = Q\Lambda Q^\top$, the regularized minimum $\tilde{w}$ satisfies:

$$\tilde{w} = Q\,(\Lambda + \alpha I)^{-1}\Lambda\, Q^\top w^*$$

- Component i is rescaled by $\dfrac{\lambda_i}{\lambda_i + \alpha}$
- High curvature ($\lambda_i \gg \alpha$): weight is barely changed
- Low curvature ($\lambda_i \ll \alpha$): weight is shrunk toward zero

L2 preserves directions important to the loss and discards directions that are not. It never sets weights to **exactly** zero.

L2 Regularization: Geometric View

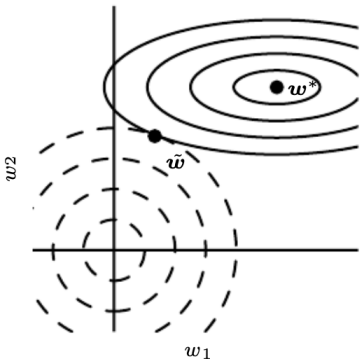

Figure: Geometry of L2 Regularization

Reading the figure:

- The solid ellipses are contours of the unregularized objective J. They are elongated because the Hessian has unequal eigenvalues.
- The dashed circles are contours of the L2 penalty $\frac{1}{2}\|w\|_2^2$.
- $\tilde{w}$ is where the two objectives reach equilibrium.

Intuition:

- Along w_1 (low curvature, small λ_1): J does not increase much, so the regularizer dominates and pulls $\tilde{w}_1$ toward zero.
- Along w_2 (high curvature, large λ_2): J is sensitive, so the regularizer has little effect and $\tilde{w}_2 \approx w_2^*$.

L1 Regularization: Sparsity

L1 penalty and regularized objective

$$\Omega(w) = \|w\|_1 = \sum_i |w_i|, \qquad \tilde{J}(w) = J(w) + \alpha \|w\|_1$$

Sub-gradient:

$$\nabla_w \tilde{J} = \alpha \, \text{sign}(w) + \nabla_w J$$

The regularization contribution is a *constant* $\pm\alpha$ per weight — it does not scale with $|w_i|$. **Optimal solution** (quadratic approximation, diagonal Hessian H):

$$\tilde{w}_i = \text{sign}(w_i^*) \max\left(|w_i^*| - \frac{\alpha}{H_{ii}}, \, 0 \right)$$

Two cases (assuming $w_i^* > 0$):

1. $|w_i^*| \leq \frac{\alpha}{H_{ii}}$: the regularizer overwhelms the objective; the optimal value is $\tilde{w}_i = 0$ (feature is discarded).
2. $|w_i^*| > \frac{\alpha}{H_{ii}}$: the weight is shifted toward zero by $\frac{\alpha}{H_{ii}}$ but remains nonzero.

Key distinction from L2

L1 can set weights to **exactly zero**. L2 only shrinks them. L1 produces **sparse** solutions and acts as a **feature selection** mechanism (LASSO, Tibshirani 1995).

L1 vs. L2 Regularization: Comparison

	L2 (Weight Decay)	**L1 (LASSO)**
Penalty Ω	$\frac{\alpha}{2}\|w\|_2^2$	$\alpha\|w\|_1$
Gradient contrib.	αw_i (proportional to w_i)	α sign(w_i) (constant magnitude)
Effect on weights	Shrinks all weights proportionally	Sets some weights to **exactly zero**
Sparsity	No	Yes — feature selection
Bayesian prior	Gaussian ($e^{-\frac{\alpha}{2}\|w\|^2}$)	Laplace ($e^{-\alpha\|w\|_1}$)
Also known as	Ridge regression, Tikhonov	LASSO
Best when	All features somewhat relevant	Many irrelevant features

In deep learning, L2 (weight decay) is most commonly used. L1 is preferred when sparse solutions or explicit feature selection is needed.

Dataset Augmentation

Core Idea

The best way to improve generalization is to train on more data. When real data is limited, create **synthetic training examples** by applying label-preserving transformations to existing ones.

Why it works

- Many tasks require invariance to transformations.
- Augmentation explicitly teaches the model these invariances.
- The effective size of the training set increases without collecting new data.

Common image transformations

- Translation (small shifts)
- Horizontal flipping
- Rotation and scaling
- Color jitter (brightness, contrast)
- Random cropping
- Gaussian noise

Dataset Augmentation: Practical Considerations

Critical Caveat

Apply only transformations that **preserve the correct label**.

- Horizontal flipping is reasonable for *dog* images.
- In OCR, flipping may change the character identity.
- A 180° rotation can transform a 6 into a 9.

Noise Injection as Augmentation

- Adding noise to inputs improves robustness.
- Noise in hidden units is often more effective because it perturbs multiple levels of representation.
- **Dropout** can be interpreted as multiplying hidden activations by binary noise.

Controlled Experiments

- When comparing algorithms, use the same augmentation strategy for all methods.
- Otherwise improvements may be due to the augmentation rather than the algorithm itself.

Early Stopping: Motivation

The observation:

When training a large model long enough:

- Training error decreases monotonically
- Validation error initially decreases, then *rises* again (U-shaped curve)

This behavior is very reliable (see Figure 7.3).

The solution:

- Store a copy of the parameters every time validation error improves
- Stop when validation error has not improved for p consecutive evaluations ("patience")
- Return the stored parameters, not the final ones

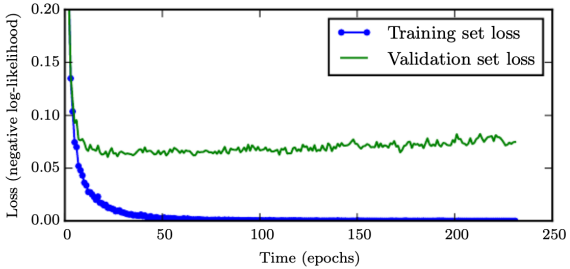

Figure: Training set loss (lower curve) vs. validation set loss (upper curve) over epochs.

Early Stopping: Algorithm

Algorithm 7.1 (Goodfellow et al., 2016):

1. Set n = steps between evaluations; p = patience; θ_0 = initial parameters
2. Initialize: $\theta \leftarrow \theta_0$, $i \leftarrow 0$, $j \leftarrow 0$, $v \leftarrow \infty$, $\theta^* \leftarrow \theta$
3. **While** $j < p$:
   - Update θ by running training for n steps; $i \leftarrow i + n$
   - $v' \leftarrow \text{ValidationSetError}(\theta)$
   - **If** $v' < v$: $j \leftarrow 0$; $\theta^* \leftarrow \theta$; $i^* \leftarrow i$; $v \leftarrow v'$
   - **Else**: $j \leftarrow j + 1$
4. Return best parameters θ^* and best step count i^*

Why early stopping is unique:

The "number of training steps" hyperparameter is swept automatically in a *single training run*. All other hyperparameters require a full re-run.

Why it is regularization:

Limiting the number of gradient steps confines the parameters to a small volume around θ_0. Under a quadratic loss approximation, gradient descent for τ steps with learning rate ϵ is equivalent to L2 regularization with:

$$\alpha \approx \frac{1}{\tau\,\epsilon}$$

More steps $\Rightarrow$ weaker effective regularization.

Early Stopping: Practical Notes (1/2)

Early stopping is one of the most commonly used regularizers in deep learning.

Advantages:

- Almost no change to the training procedure
- No modification of the objective function
- No new gradient computations
- Can be combined with other regularizers

Cost:

- Periodic validation set evaluation (can be parallelized)
- Storage of best parameters (usually negligible)

Key idea:

- Stop training when validation error stops improving

Early Stopping: Practical Notes (2/2)

After early stopping: using all the data

Two common strategies:

1. **Reinitialize and retrain:** train on full dataset for i^* steps (optimal stopping time)
2. **Continue training:** resume from early-stopped parameters on full data until reaching the same training loss

Relationship to L2 regularization

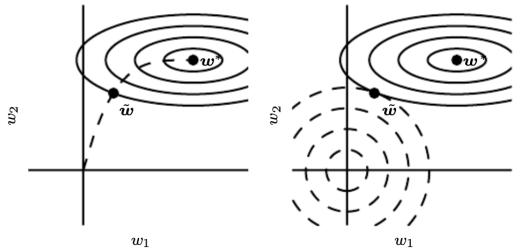

Figure: Trajectory of stochastic gradient descent under early stopping (left) compared with L2 regularization (right).

Ensemble Methods and Bagging

Model averaging

Train k different models; combine their predictions. Different models usually do not make all the same errors on the test set.

Expected squared error of the ensemble:

If each model i makes error ϵ_i with variance v and pairwise covariance c:

$$\mathbb{E}\left[\left(\frac{1}{k}\sum_i \epsilon_i\right)^2\right] = \frac{1}{k}v + \frac{k-1}{k}c$$

- Perfectly uncorrelated ($c = 0$): error $= v/k$ — decreases linearly with k
- Perfectly correlated ($c = v$): error $= v$ — no gain

Bagging (bootstrap aggregating, Breiman 1994):

1. Construct k datasets by sampling the training set *with replacement*
2. Train one model on each dataset
3. Average predictions (regression) or vote (classification)

Bagging for neural networks:

Even when training on the same dataset, differences in random initialization, minibatch sampling, and hyperparameters produce partially independent errors.

Dropout approximates bagging over **exponentially many** parameter-sharing sub-networks at negligible cost.

Dropout: Motivation and Definition

The bagging problem for large networks:

Bagging requires training and evaluating k separate models. With large neural networks, even $k = 10$ is very expensive.

Dropout's solution (Srivastava et al., 2014):

Approximate bagging over *exponentially many* sub-networks of a single base network, by randomly removing units.

Sub-networks by masking

Remove a unit by multiplying its output by zero. A network with n units has 2^n possible sub-networks. For $n = 1000$ this is 2^{1000} — far more models than could be trained explicitly.

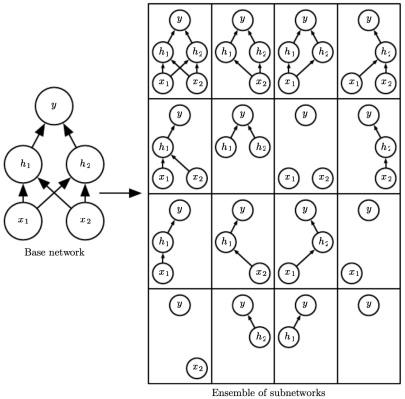

Figure: Base network (2 inputs, 2 hidden units, 1 output) and all 16 possible sub-networks formed by dropping different subsets of units.

Dropout: Training Procedure (1/2)

Each minibatch step:

1. Sample a binary mask μ (one entry per input/hidden unit)
   - ▷ Input unit kept with probability 0.8
   - ▷ Hidden unit kept with probability 0.5
2. Multiply each unit output by its mask entry
3. Run forward propagation through masked network
4. Run backpropagation through same masked network
5. Update parameters of active sub-network

Objective:

$$\min_{\theta} \mathbb{E}_{\mu}\big[J(\theta, \mu)\big]$$

A stochastic gradient is obtained by sampling a new mask μ at each step.

Key idea: Each minibatch trains a different sub-network.

Dropout: Training Procedure (2/2)

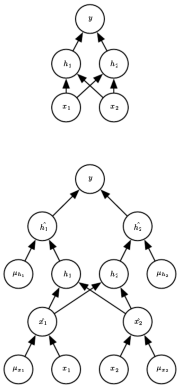

Figure: Forward propagation with dropout. Each unit is multiplied by its mask entry μ before passing to the next layer.

Why dropout works as regularization:

- Prevents co-adaptation between units
- Forces each unit to learn useful, robust features
- Equivalent to training an ensemble of sub-networks

Multiplicative noise:

- Units cannot rely on scaling to bypass dropout
- Stronger regularization than additive noise

Dropout: Inference and Weight Scaling (1/2)

The inference challenge:

At test time we want predictions from all 2^n sub-networks. Evaluating each is infeasible.

Weight scaling inference rule (Hinton et al., 2012):

Use all units, but multiply the weights by the inclusion probability p:

$$W_{\text{test}} = p \cdot W_{\text{train}}$$

For $p = 0.5$: divide all weights by 2 after training.

Why it works:

At train time, a downstream unit receives a hidden unit's signal only fraction p of the time.

At test time the unit is always present. Scaling by p matches the expected total input to each unit, keeping the expected activation the same as during training.

Alternatively, multiply activations by 2 *during training* ("inverted dropout"), so no adjustment is needed at test time.

Dropout: Inference and Weight Scaling (2/2)

Exactness of the weight scaling rule:

The rule is *exact* for:

- Softmax regression classifiers
- Linear regression with Gaussian outputs
- Linear hidden layers

For deep nonlinear networks it is an *approximation* that works very well in practice.

Alternative: geometric mean approximation

Approximate the arithmetic mean of the ensemble predictions by the geometric mean (Warde-Farley et al., 2014). This is implemented exactly by the weight scaling rule.

Monte Carlo sampling:

Even averaging 10–20 random masks at test time gives good performance, and can outperform the weight scaling approximation on some problems (Gal and Ghahramani, 2015).

Dropout: Practical Considerations (1/2)

Computational cost:

- Training: $O(n)$ overhead per example (mask sampling + multiplication)
- Memory: $O(n)$ to store mask during backprop
- Test time: no extra cost (after weight scaling)

Model size: Dropout reduces effective capacity, so larger models are typically required.

In practice:

- Lower validation error is achievable
- But requires more parameters and training iterations

When dropout is less effective:

- Very large datasets (regularization less needed)
- Very small datasets (< 5000 examples); Bayesian methods may outperform
- Very thin or narrow layers

Dropout: Practical Considerations (2/2)

Comparison with weight decay:

- Linear regression: dropout $\approx$ L2 regularization (feature-dependent scaling)

  (Wager et al., 2013)
- Deep nonlinear networks: not equivalent

Dropout vs. other regularizers:

- Often outperforms weight decay, norm constraints, and sparsity penalties as standalone methods
- Strong empirical performance (Srivastava et al., 2014)

Variants:

- DropConnect: drops individual weights instead of units

  (Wan et al., 2013)

Dropout works well for models with distributed representations trained using SGD.

Label Smoothing and Noise Robustness

The problem with hard targets:

A softmax + cross-entropy classifier is trained to output probability 1 for the correct class. The softmax can only approach 1 asymptotically (by making the correct class logit $\to +\infty$). The model will chase ever-larger weights and may never converge.

Consequence:

- Overconfident, poorly calibrated predictions
- Weights grow without bound (unless other regularization is applied)

Label smoothing (Szegedy et al., 2015)

Replace the hard one-hot target with a soft target:

$$y_k^{\text{smooth}} = \begin{cases} 1-\epsilon & \text{correct class} \\ \dfrac{\epsilon}{K-1} & \text{other classes} \end{cases}$$

where ϵ is small (e.g. 0.1) and K is the number of classes.

Effect:

- The model cannot assign probability exactly 1 to any class — the impossible target is removed
- Prevents pursuit of hard probabilities without discouraging correct classification
- Improves calibration: model confidence better reflects accuracy

Module 6 Summary: Regularization (1/2)

- **Regularization** reduces generalization error by trading increased bias for reduced variance. The best deep learning model is a **large, regularized** model.
- **L2 (weight decay):** $\tilde{J} = J + \frac{\alpha}{2}\|w\|^2$. Multiplicatively shrinks weights each step. Rescales w^* by $\lambda_i/(\lambda_i + \alpha)$. Never produces exact zeros. Equivalent to a Gaussian prior.
- **L1 regularization:** $\tilde{J} = J + \alpha\|w\|_1$. Constant-magnitude sub-gradient. Sets small weights to **exactly zero** (sparsity). Feature selection (LASSO). Equivalent to a Laplace prior.
- **Dataset augmentation:** generate new (x, y) pairs by applying label-preserving transformations. Highly effective for vision and speech.

Module 6 Summary: Regularization (2/2)

- **Early stopping:** halt training at the lowest validation error. Equivalent to L2 regularization with $\alpha \approx 1/(\tau\epsilon)$. Probably the most widely used regularizer in deep learning. Free: requires no change to the objective.
- **Ensemble methods / Bagging:** train k models on bootstrap samples; combine predictions. Expected error $\leq v/k$ when errors are uncorrelated. Dropout approximates a bagged ensemble of 2^n sub-networks.
- **Dropout:** randomly mask units each minibatch step. Prevents co-adaptation. Use weight scaling ($\times p$) at test time. Requires a larger model to compensate for reduced effective capacity.
- **Label smoothing:** replace hard one-hot targets with soft targets to prevent overconfident predictions and improve calibration.

How Learning Differs from Pure Optimization

Pure optimization

Minimize $J(\theta)$ as a goal in itself. Done when a (local) minimum is reached. The objective equals the goal.

Machine learning optimization

Minimize training loss $J(\theta)$ as a *proxy* for true goal: low generalization error $J^*(\theta) = \mathbb{E}_{p_{\text{data}}}[L]$.

Key difference: we optimize the **empirical risk** (training set average) and hope the true risk also decreases.

Four ways learning differs:

1. **We don't minimize to zero.** Training halts via early stopping (when validation error stops improving), not when the gradient vanishes.
2. **Surrogate losses.** We minimize NLL (has useful gradients everywhere) instead of 0-1 loss (gradient is zero or undefined almost everywhere).
3. **Continued improvement.** Test 0-1 loss often keeps decreasing after training 0-1 loss hits zero, as NLL continues pushing classes apart.
4. **Non-convex loss surfaces.** No convex guarantees; many local minima and saddle points.

Batch, Minibatch, and Stochastic Gradient Descent

The gradient as an expectation:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x,y\sim\hat{p}_{\text{data}}}[\nabla_\theta L]$$

This expectation can be estimated from a random *subset* of examples.

Batch	All m examples. Cost $O(m)$ per step. Accurate. Slow.
Minibatch	$m' \ll m$ examples. Cost $O(m')$. Noisy. Fast.
Online	1 example. Very noisy. Streaming data.

Standard error of gradient estimate:

$\text{SE} = \sigma/\sqrt{m'}$. Using $100\times$ more examples reduces SE by only $10\times$ — less than linear returns.

Practical minibatch sizes:

- Multicore GPUs: powers of 2 (32–256 typical)
- Too small: poor GPU utilization
- Too large: memory limits; less implicit regularization
- Small batches add noise that regularizes (Wilson and Martinez, 2003)

Always **randomly shuffle** training data before constructing minibatches. Ordered data causes correlated batches and biased gradient estimates.

Optimization Challenge: Ill-Conditioning (1/2)

Ill-conditioning of the Hessian H

When the condition number

$$\kappa(H) = \frac{\lambda_{\max}}{\lambda_{\min}}$$

is large, the loss surface is much more curved in some directions than others.

Effect on gradient descent:

A gradient step $-\epsilon g$ changes the cost approximately as:

$$-\epsilon g^\top g + \tfrac{1}{2}\epsilon^2 g^\top H g$$

If $g^\top H g \gg g^\top g$, even a small step can increase the cost, forcing very small learning rates.

Key intuition: Ill-conditioning creates narrow, curved valleys where optimization is inefficient.

Consequence: Gradient descent progresses slowly because step sizes must be small in all directions.

Typical symptoms:

- Oscillations across narrow valleys
- Slow convergence
- Sensitive learning rate tuning

Optimization Challenge: Ill-Conditioning (2/2)

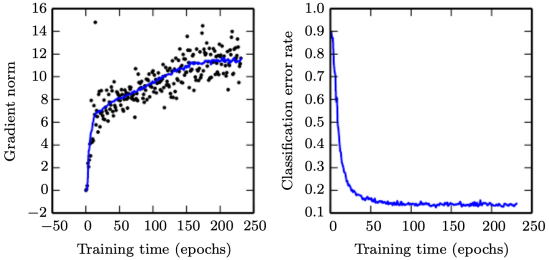

Figure: Gradient norm and classification error during training of a convolutional network. The gradient norm grows throughout training, yet training still succeeds.

Key observation: Even when gradient norms grow, optimization can still succeed.

Why this matters: Large gradients do not necessarily indicate instability—geometry of the loss surface is the real issue.

Solutions:

- Momentum
- Adaptive learning rates (RMSProp, Adam)
- Second-order methods

Optimization Challenge: Local Minima

Why neural networks have many local minima:

Weight space symmetry: permuting hidden units gives an equivalent model. A network with m layers of n units each has $n!^m$ equivalent local minima.

Are local minima a problem?

Local minima with *high cost* are problematic. However, for sufficiently large networks:

- Most local minima have *low cost* (Dauphin et al., 2014; Goodfellow et al., 2015)
- It is not important to find the global minimum, just a point with low generalization error

How to diagnose:

Plot the gradient norm over time. If the gradient norm does not shrink to near zero, the problem is *not* local minima. Neural networks often do not reach a region of small gradient at all.

Saddle points are more common:

In high-dimensional spaces, at a saddle point the Hessian has both positive and negative eigenvalues. As dimension grows, obtaining all-positive eigenvalues (a local minimum) becomes exponentially unlikely.

First-order methods typically escape saddle points empirically. Newton's method can jump to them.

Optimization Challenge: Cliffs and Exploding Gradients (1/2)

Exploding gradients and cliffs:

Deep networks multiply many weight matrices. In regions where weights become large, gradients can grow rapidly, leading to instability.

A single gradient step can jump across a sharp "cliff" in the loss surface, causing large parameter changes.

Gradient clipping:

If the gradient norm exceeds a threshold v, rescale:

$$g \leftarrow g \cdot \frac{v}{\|g\|}$$

This prevents excessively large updates.

Key root cause (RNN intuition):

$$W^t = V \operatorname{diag}(\lambda)^t V^{-1}$$

- Eigenvalues $|\lambda| > 1 \rightarrow$ explosion
- Eigenvalues $|\lambda| < 1 \rightarrow$ decay

This explains why recurrent networks are especially unstable.

Optimization Challenge: Cliffs and Vanishing Gradients (2/2)

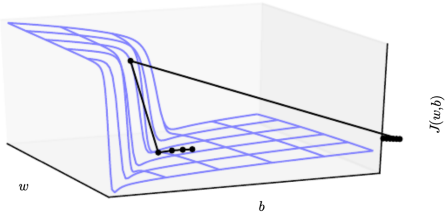

Figure: Sharp nonlinear cliff in the loss surface. A gradient step near the cliff can catapult parameters very far.

Vanishing gradients:

- Gradients shrink exponentially through many layers
- Early layers receive almost no learning signal
- Training becomes extremely slow or stalls

Solutions:

- ReLU activations
- Careful initialization (Xavier/He)
- Residual (skip) connections
- LSTM gating for RNNs

SGD: Algorithm (Algorithm 8.1)

Stochastic Gradient Descent — Update at training iteration k

Require: learning rate ϵ_k. **Require:** initial parameters θ.

While stopping criterion not met:

1. Sample a minibatch of m examples $\{x^{(1)}, \ldots, x^{(m)}\}$ with targets $y^{(i)}$
2. Compute gradient estimate:

$$\hat{g} \leftarrow +\frac{1}{m} \nabla_\theta \sum_i L\Big(f(x^{(i)}; \theta),\, y^{(i)}\Big)$$

3. Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{g}$

Why the learning rate must decay:

Even at the true minimum, $\hat{g} \neq 0$ (minibatch noise). A fixed ϵ causes perpetual bouncing. Decaying ϵ_k forces convergence.

Sufficient convergence conditions:

$$\sum_{k=1}^{\infty} \epsilon_k = \infty \qquad \sum_{k=1}^{\infty} \epsilon_k^2 < \infty$$

Linear decay schedule:

$$\epsilon_k = (1-\alpha)\epsilon_0 + \alpha\, \epsilon_\tau, \quad \alpha = \tfrac{k}{\tau}$$

After iteration τ, hold ϵ constant at $\epsilon_\tau \approx 1\%\, \epsilon_0$.

Momentum: Algorithm (Algorithm 8.2)

SGD with Momentum

Require: learning rate ϵ, momentum α. **Require:** initial θ, initial velocity v.

While stopping criterion not met:

1. Sample minibatch $\{x^{(1)}, \ldots, x^{(m)}\}$ with targets $y^{(i)}$
2. Compute gradient estimate:

$$g \leftarrow \frac{1}{m} \nabla_\theta \sum_i L\Big(f(x^{(i)}; \theta),\, y^{(i)}\Big)$$

3. Compute velocity update:

$$v \leftarrow \alpha v - \epsilon\, g$$

4. Apply update: $\theta \leftarrow \theta + v$

Motivation:

- Accumulates an exponentially decaying moving average of past gradients
- Damps oscillations across narrow valleys (gradients cancel)
- Accelerates progress along consistent directions (gradients add)

Terminal velocity (constant gradient g):

$$\|v_{\text{terminal}}\| = \frac{\epsilon \|g\|}{1 - \alpha}$$

Typical values of α are 0.5, 0.9, and 0.99. Using $\alpha = 0.9$ often speeds up SGD by about $10\times$. It can be interpreted as a hockey puck sliding on ice with friction (momentum smoothing the motion).

Nesterov Momentum: Algorithm (Algorithm 8.3)

SGD with Nesterov Momentum (Sutskever et al., 2013)

Require: learning rate ϵ, momentum α. **Require:** initial θ, initial velocity v.

While stopping criterion not met:

1. Sample minibatch $\{x^{(1)}, \ldots, x^{(m)}\}$ with labels $y^{(i)}$
2. Apply interim update ("look-ahead"):

$$\tilde{\theta} \leftarrow \theta + \alpha v$$

3. Compute gradient at interim point:

$$g \leftarrow \frac{1}{m} \nabla_{\tilde{\theta}} \sum_i L\Big(f(x^{(i)}; \tilde{\theta}), y^{(i)}\Big)$$

4. Compute velocity update:

$$v \leftarrow \alpha v - \epsilon g$$

5. Apply update: $\theta \leftarrow \theta + v$

Key difference from standard momentum:

Standard momentum evaluates the gradient at the current position θ, then moves.

Nesterov momentum first moves to the interim position $\tilde{\theta} = \theta + \alpha v$, then evaluates the gradient there. This acts as a *correction*: slows down when the model has overshot.

Convergence (convex, batch):

Reduces excess error from $O(1/k)$ to $O(1/k^2)$ (Nesterov, 1983).

Stochastic case: no improvement in convergence rate, but often better empirically.

Parameter Initialization: Why It Matters

Initialization determines:

- Whether training converges at all
- How quickly convergence occurs
- Which local minimum is reached
- The generalization error of the final model

The symmetry-breaking requirement:

If two hidden units have identical initial parameters and the same inputs, they receive identical gradient updates forever — they remain identical (redundant).

Solution: always initialize weights *randomly* to break symmetry. Biases may be initialized to zero.

Competing pressures on weight scale:

- **Too small:** activations shrink exponentially layer by layer; gradients vanish during backpropagation; early layers cannot learn
- **Too large:** activations or gradients explode; sigmoid/tanh units saturate, losing all gradient; numerical instability
- **Just right:** activations and gradients maintain roughly stable magnitudes across all layers

Bias initialization

Set biases to zero (default). For ReLU hidden units, consider 0.1 to avoid dead units at initialization. For output units, initialize to match the marginal statistics of the target.

Parameter Initialization: Strategies

Glorot / Xavier initialization (Glorot and Bengio, 2010)

For a layer with m inputs and n outputs:

$$W_{ij} \sim U\left(-\sqrt{\frac{6}{m+n}},\ \sqrt{\frac{6}{m+n}}\right)$$

Goal: keep activation variance and gradient variance approximately equal across all layers (derived for linear networks).

He initialization (He et al., 2015)

For ReLU networks (accounts for the factor-of-2 reduction from zeroing negative inputs):

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{m}\right)$$

Other strategies:

- **Orthogonal initialization** (Saxe et al., 2013): initialize weight matrices as random orthogonal matrices with a gain factor g for the nonlinearity. All singular values equal 1 at initialization.
- **Sparse initialization** (Martens, 2010): each unit has exactly k non-zero incoming weights (drawn from a Gaussian). Keeps total input magnitude independent of layer size.

Practical advice:

- Treat weight scale as a hyperparameter
- Monitor range of activations and gradients on the first minibatch
- Increase scales layer by layer until all layers have reasonable activation ranges

AdaGrad: Algorithm (Algorithm 8.4)

AdaGrad (Duchi et al., 2011) — Individually adapted learning rates

Require: global learning rate ϵ, small constant $\delta \approx 10^{-7}$. **Require:** initial θ.

Initialize: gradient accumulation $r = 0$.

While stopping criterion not met:

1. Sample minibatch $\{x^{(1)}, \ldots, x^{(m)}\}$ with targets $y^{(i)}$
2. Compute gradient:

$$g \leftarrow \frac{1}{m} \nabla_\theta \sum_i L\Big(f(x^{(i)}; \theta), y^{(i)}\Big)$$

3. Accumulate squared gradient:

$$r \leftarrow r + g \odot g$$

4. Compute update:

$$\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{r}} \odot g$$

Effect:

- Parameters with large gradients: r grows fast $\Rightarrow$ effective learning rate shrinks fast
- Parameters with small gradients: r stays small $\Rightarrow$ effective learning rate stays larger
- Progress is equalized across directions in parameter space

Problem: r keeps accumulating, so the learning rate can shrink too fast, causing training to stall early in non-convex settings.

Works well for: convex problems.

Less suitable for: deep networks with non-convex loss surfaces.

RMSProp: Algorithm (Algorithm 8.5)

RMSProp (Hinton, 2012) — Exponential moving average of squared gradients

Require: global lr ϵ, decay rate ρ, constant $\delta \approx 10^{-6}$. **Require:** initial θ.

Initialize: accumulation variable $r = 0$.

While stopping criterion not met:

1. Sample minibatch $\{x^{(1)}, \ldots, x^{(m)}\}$ with targets $y^{(i)}$
2. Compute gradient:

$$g \leftarrow \frac{1}{m} \nabla_\theta \sum_i L\Big(f(x^{(i)}; \theta),\, y^{(i)}\Big)$$

3. Accumulate squared gradient (exponential moving average):

$$r \leftarrow \rho\, r + (1 - \rho)\, g \odot g$$

4. Compute update: $\Delta\theta = -\frac{\epsilon}{\sqrt{\delta + r}} \odot g$
5. Apply update: $\theta \leftarrow \theta + \Delta\theta$

How RMSProp fixes AdaGrad:

AdaGrad's r never decreases (accumulates the entire history). RMSProp's r tracks only recent gradient magnitudes: old gradients are exponentially forgotten.

In smoother regions, r decreases, allowing the learning rate to increase again. AdaGrad cannot adapt this way.

New hyperparameter: ρ (decay rate, typically 0.9) controls the time window of the moving average.

Status: one of the most widely used optimizers for deep learning in practice.

RMSProp with Nesterov Momentum (Algorithm 8.6) (1/2)

RMSProp + Nesterov Momentum (Algorithm 8.6)

Require: learning rate ϵ, decay rate ρ, momentum α, initial θ, initial velocity v

Initialize:

$$r = 0$$

Each iteration:

1. Sample minibatch
2. Look-ahead step:

$$\tilde{\theta} = \theta + \alpha v$$

3. Compute gradient at $\tilde{\theta}$:

$$g = \nabla_{\tilde{\theta}} L$$

4. Accumulate squared gradients:

$$r = \rho r + (1 - \rho) g \odot g$$

Velocity update:

$$v = \alpha v - \frac{\epsilon}{\sqrt{r}} \odot g$$

$$\theta = \theta + v$$

Key idea: Combine look-ahead momentum with adaptive per-parameter scaling.

RMSProp with Nesterov Momentum (Algorithm 8.6) (2/2)

What this combines:

- **RMSProp:** adaptive learning rates via moving average of g^2
- **Nesterov momentum:** gradient evaluated at look-ahead position

Why it works:

- Momentum smooths noisy updates
- RMSProp rescales parameters with different gradient magnitudes
- Look-ahead improves correction quality

When to use:

- Ill-conditioned loss surfaces
- Sparse or heterogeneous gradients
- Deep networks with unstable training dynamics

Note: Element-wise scaling by $\frac{1}{\sqrt{r}}$ adapts step sizes per parameter.

Adam: Algorithm (Algorithm 8.7) (1/2)

Adam (Kingma & Ba, 2014; Goodfellow et al., Algorithm 8.7)

Require: learning rate $\epsilon = 0.001$, $\rho_1 = 0.9$, $\rho_2 = 0.999$, $\delta = 10^{-8}$, initial θ

Initialize:

$$s = 0, \quad r = 0, \quad t = 0$$

Each iteration:

1. Sample minibatch, compute gradient g
2. $t \leftarrow t + 1$
3. $s \leftarrow \rho_1 s + (1 - \rho_1) g$
4. $r \leftarrow \rho_2 r + (1 - \rho_2) g \odot g$

Bias correction:

$$\hat{s} = \frac{s}{1 - \rho_1^t}, \quad \hat{r} = \frac{r}{1 - \rho_2^t}$$

Update rule:

$$\Delta\theta = -\epsilon \frac{\hat{s}}{\sqrt{\hat{r}} + \delta}$$

$$\theta \leftarrow \theta + \Delta\theta$$

Adam: Algorithm (Algorithm 8.7) (2/2)

Connection to RMSProp + momentum:

- s: momentum (smoothed gradient)
- r: adaptive scaling (second moment)
- bias correction removes early underestimation

Effective learning rate:

$$\epsilon_{\text{eff}} \approx \frac{\epsilon}{\sqrt{\hat{r}}}$$

Properties:

- Robust to hyperparameters
- Works well out-of-the-box
- Fast convergence in practice
- Sometimes slightly worse generalization than SGD+momentum

Key idea: Adaptive step sizes + momentum = stable and fast optimization.

Choosing an Optimizer

No universal best optimizer.

Schaul et al. (2014) compared many algorithms across many tasks. The family of adaptive-rate methods (RMSProp, AdaDelta) performed robustly, but no single algorithm dominated all tasks.

Currently popular choices:

Optimizer	When to use
SGD + momentum	Best generalization if you can tune ϵ carefully
RMSProp	General purpose; especially RNNs
Adam	Robust default; little tuning required
AdamW	Adam with decoupled weight decay

Practical guidance:

- Start with **Adam** at $\epsilon = 10^{-3}$. It usually works with minimal tuning.
- If best generalization is needed and compute is available, try **SGD + momentum** with a learning rate schedule (cosine decay or linear decay).
- **The learning rate** is the single most important hyperparameter to tune for any optimizer.
- Monitor both training and validation loss. Violent oscillations $\Rightarrow$ reduce ϵ. Very slow progress $\Rightarrow$ increase ϵ or use warmup.

Improvements from architecture, regularization, or more data typically dwarf improvements from optimizer choice.

Module 7 Summary: Optimization (1/2)

- **Learning $\neq$ pure optimization:** minimize a surrogate training loss as proxy for generalization error. Halt at lowest validation error, not zero gradient. NLL is preferred over 0-1 loss (useful gradients everywhere).
- **Minibatch SGD:** gradient is an expectation; estimate from m' examples. Cost $O(m')$ independent of training set size. Shuffle data; typical batch 32–256. Small batches add implicit regularization.
- **ill-conditioning:** large $\kappa(H)$ forces small learning rates and causes oscillation. Addressed by momentum and adaptive rates.
- **Local minima and saddle points:** local minima are mostly low-cost for large networks. Saddle points are common in high dimensions but escapable by first-order methods. Gradient clipping prevents cliff-induced instability.

Module 7 Summary: Optimization (2/2)

- **SGD:** core update $\theta \leftarrow \theta - \epsilon \hat{g}$. Decay ϵ_k over time. Linear schedule: $\epsilon_\tau \approx 1\%$ of ϵ_0.
- **Momentum:** $v \leftarrow \alpha v - \epsilon g$; $\theta \leftarrow \theta + v$. Damps oscillations; terminal speed $= \epsilon \|g\| / (1 - \alpha)$. Nesterov variant evaluates gradient at the look-ahead position $\theta + \alpha v$.
- **Initialization:** random weights break symmetry. Glorot: $U(-\sqrt{6/(m+n)}, \sqrt{6/(m+n)})$. He: $\mathcal{N}(0, 2/m)$ for ReLU. Scale is a hyperparameter.
- **AdaGrad:** $r \leftarrow r + g \odot g$; lr scales as $1/\sqrt{r}$. Shrinks too fast for non-convex settings.
- **RMSProp:** $r \leftarrow \rho r + (1 - \rho) g \odot g$. Exponential moving average fixes AdaGrad's forgetting problem. Widely used.
- **Adam:** combines momentum (s) and RMSProp (r) with bias correction. Robust default. Start at $\epsilon = 10^{-3}$.

Questions?

Contact Information	
Author	Emmanuel Djegou, PhD
Topic Area	Data Science, Statistical Modeling & Survival Analysis
Email	emmanueldjegou5@gmail.com
LinkedIn	linkedin.com/in/emmanuel-djegou-phd-5652b2254